

Don't see what you're looking for? Log in for the full Help Center experience

How Policies work

Updated 5 months ago

Overview

Understanding how the Policy Engine evaluates and applies rules is essential for creating effective Policies. This article explains the core concepts that govern how your Policies work, including the first-match principle, rule ordering, and how transactions are matched against your rules.

Policy rules

Policy rules have two components that you can define:

- **Parameters:** Define which characteristics a transaction must “match” for the rule to apply.
 - **Scope:** Who can initiate it? Where can it go to or from? Which asset is the rule for?
- **Actions:** Define what will happen next out of the following options:
 - **Allow:** The Policy Engine automatically approves and forwards to a designated signer.
 - **Approved by:** Other users or user groups must approve or deny the allowed transaction. All of the associated user roles must be [authorized approver user roles](#).
 - **Block:** The Policy Engine automatically blocks the transaction.

A sample rule is below. Your Policies include multiple ordered rules similar to this one. The Policy Engine compares transactions against your Policies before you can sign them to the blockchain.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

The first-match principle

Policies operate according to the first-match principle. For each submitted transaction, the Policy Engine scans the appropriate Policy from top to bottom and applies the first rule that it matches. The scan stops after a successful match. The top rule is checked first. If it is a match, it performs the rule's action. If not, rule two is checked, and so on.

Due to the first-match principle, the order of your rules is important for receiving the intended outcomes. For example, you may not want to place a rule that allows any transfer in any amount at the top of your Transfer Policy since most transactions would be allowed without needing to meet the stricter requirements in the other rules. Rules with overlapping parameters should be ordered so that higher priority rules are scanned first.

For security, the [default block-all rule](#) is always the last rule at the bottom of each Policy. If no other rule matches a transaction, the block-all rule blocks it.

Example: Policy Engine applies the first-match principle to a transaction

John Smith, a trader at company XYZ, submits a transaction. In his company's workspace, he is assigned the following workspace role and user group:

- **Role:** Signer, authorized to sign transactions.
- **User group:** Traders, a custom group that John's company created in their workspace. The group has all traders with Signer roles.

John submits a transfer request for \$500,000 of Bitcoin from his vault account to one of his whitelisted counterparties. His company's Transfer Policy has the four rules shown below:

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/

01	MG Only	Management	Initiator	Any	Any	\$1M per transaction	All	Require approval of: 1 of 1 user
02	Over \$1M	Traders	Initiator	Any	Any Whitelisted address	\$1M per transaction	All	Require approval of: 1 of 3 users
03	Trading Daily	Traders Mgt	Initiator	Any	Any	\$0M per transaction	All	Require approval of: 5 of 5 users
04	Default transfer rule	Any	Initiator	Any	Any	Unlimited USD per transaction	All	Block

After John submits the transfer request, the Policy Engine:

1. *Skips* rule one, which only applies when a Management group user is Initiator.
2. *Skips* rule two, which only applies to transactions greater than \$1 million.
3. *Applies* rule three because the transaction matches each of its parameters.
4. Sends the request to the approver that is defined under **Action**. Select **Require approval of** to see the name of the designated approver. For this rule, it is the Head of Trading.
5. After the Head of Trading approves the transaction, a signing request is sent to the Designated Signer.

Since rule three sets Initiator as Designated Signer, John can then sign the transaction himself. Read [Policy examples](#) to learn how to create tailored sets of rules for various common business use cases.

Ordering your rules

It's important to put rules in order from the most restrictive rules to the least restrictive rules to correctly apply the [first-match principle](#). The order of rules can affect whether or not the Policy Engine properly enforces your Policy rules.

For example:

- Put time period-based rules before single-transaction rules. Doing this ensures that time-based restrictions get enforced. [Learn more about creating your Policy](#).

Transfer policy								
Published by Liam Morgan on May 18, 2025 at 20:48.								
#	Name	Initiator	Signer	Source	Destination	Amount	Asset	Action
01	Vault-to-vault	Anyone	Initiator	Any	Any vault account Whitelisted address	\$10,000,000 in 8 hours	All	Require approval of: 1 of 1 group
02	Funding	Admin	Initiator	Any	Any	Unlimited USD per transaction	All	Block

Don't see what you're looking for? Log in for the full Help Center experience



Fireblocks Help Center



- Rule two has an amount range with a \$10 million minimum, no maximum, and needs *two* approvals.

#	Name	Initiator	Signer	Source	Destination	Amount	Asset	Action
01	Vault-to-vault	Anyone	Initiator	@ Any	@ Any vault account Initiated address	\$10,000,000 in 8 hours	All	Require approval of: 1 of 2 group
02	Funding	Admin	Initiator	@ Any	@ Any	\$1,000,000 per transaction	All	Require approval of: 1 of 1 group

- List the more restrictive rule (rule two) first so the Policy Engine enforces its stricter threshold and approval flow. If not, rule two won't get enforced. A specific example:
 - If *rule one is listed first*, a \$20 million transaction would match it and only need one approval. However, the intent of rule two is that a transaction that big needs two approvers. If you *list rule two first*, you don't have this issue.

Using user groups in Policy rules

User groups enable you to apply rules to sets of multiple users without needing to add new rules when the users change. This allows you to simplify your Policies and use groups for key actions like initiating and approving transactions.

You can also define approval quorums within a group as part of the **Approved by** action for a rule. For example, you can make a rule that requires any two members of a user group called Management to approve if a transaction matches that rule's parameters.

Two more ways you can use user groups in rules:

- The workspace Owner can initiate a certain type of transfer, but at least one other member of the Admins user group must approve.
- Any member of the Admins user group can initiate a certain type of transfer.

[Learn more about User group management and best practices](#) for detailed guidelines and steps to create and manage user groups.

Don't see what you're looking for? Log in for the full Help Center experience



Fireblocks Help Center



- [Policy Best Practices](#) - Follow recommendations for effective Policy management
- [Policy Rule Parameters](#) - Reference guide for all available parameters
- [Policy Rule Examples](#) - Review some common use cases
- [Policy Rule Error Messages](#) - Troubleshoot error messages when creating or editing rules

Was this article helpful?

Yes

No

ON THIS PAGE

Overview

Policy rules

The first-match principle

Ordering your rules

Using user groups in Policy rules

Next steps



[fireblocks.com](#)

[Status](#)

[API Docs](#)

[Console](#)

[info@fireblocks.com](#)



Fireblocks © 2026. All Rights Reserved. NMLS Registration Number: 2066055

[Privacy Policy](#)